

```
<meta name="viewport" content="width=device-width; initial-scale=1.0">
```

Here, setting the initial scale to 1 overrides the default image resizing feature.

Media queries

CSS3 has introduced a powerful tool in the form of 'media queries', which actually allows Web developers to inspect the physical characteristics of the device rendering their work.

For example, let's look at the following query:

```
<link rel="stylesheet" type="text/css"
media="screen and (max-device-width: 480px)"
href="smaller-res.css" />
```

Here, we are querying is *equal to or less than 480px*, so load *smaller-res.css*; otherwise, just ignore the link altogether.

Now, a media query is not only restricted to the 'links'; tag but can also be included in CSS either as the `@media` rule or as a `@import` directive.

Here are sample code snippets for the same:

```
@media:
@media screen and (max-device-width: 480px) {
  .column {
    float: none;
  }
}

@import:
@import url("shetland.css") screen and (max-device-width:
480px);
```

In conclusion, media queries are conditional comments. By using them, rather than targeting specific versions of the browser, we can correct issues in our layout.

JavaScript

Another method to create a RWD is to use JavaScript, especially for the devices that don't support CSS3 media queries. Shown below is a simple jQuery snippet that detects the browser's width and changes the style sheet accordingly:

```
<script type="text/javascript" src="http://ajax.googleapis.
com/ajax/libs/jquery/1.4.4/jquery.min.js"></script>
<script type="text/javascript">
$(document).ready(function(){
    $(window).bind("resize", resizeWindow);
    function resizeWindow(e){
        var newWindowWidth = $(window).width();
```

```
// If width width is below 600px, switch to the mobile
stylesheet

    if(newWindowWidth < 600){
$("link[rel=stylesheet]").attr({href : "mobile.css"});
} // Else if width is above 600px, switch to the large
stylesheet
    else if(newWindowWidth > 600){
$("link[rel=stylesheet]").attr({href : "style.
css"});
}
}
});
</script>
```

Showing and hiding content

Creating responsive Web designs should not only be about porting content and making everything fit on every view port. But as developers, at times, we do want the choice of being able to hide some content in smaller resolution screens compared to what is being displayed on a bigger layout. Fortunately, CSS provides a simple *display* tag to hide a block of HTML content.

We can declare *display: none* for a HTML block in a specific style sheet, or detect browser width and do it through JavaScript. Also, the other way around, we can hide some content that is specific to mobile devices from bigger screens. Refer to the References section for an example of the same.

I have tried giving an overview of the problems that the Web development world is facing and the approaches to countering these challenges. There is a lot of work happening in this area, and this gives everyone an opportunity to test their mettle. 

References

- [1] http://www.w3schools.com/html/html_responsive.asp
- [2] <http://alistapart.com/article/responsive-web-design/>
- [3] <http://www.smashingmagazine.com/2011/01/guidelines-for-responsive-web-design/>
- [4] Git Hub Location for Responsive Image Files: <https://github.com/filamentgroup/Responsive-Images>
- [5] https://en.wikipedia.org/wiki/Responsive_web_design
- [6] <http://alistapart.com/article/fluid-images>
- [7] <http://www.alistapart.com/articles/fluidgrids/>
- [8] <http://unstoppablerobotninja.com/entry/fluid-images>
- [9] <http://zomigi.com/blog/hiding-and-revealing-portions-of-images/>
- [10] <http://zomigi.com/blog/hiding-and-revealing-portions-of-images/>
- [11] Showing Hidden content example: <https://www.smashingmagazine.com/2011/01/guidelines-for-responsive-web-design/#showing-or-hiding-content>

By: Ashish Bunkar

The author is a Linux professional with more than five years' experience. His interests are SAN and virtualisation technologies.